

Rapport : Résolution du Problème NP-Difficile de Kemeny.

Goales Saphiya et Teibi Zahra

Numéro étudiant : 21233538 et 21211965

Table des matières

1	Introduction	2
2	Préliminaire	2
3	Réduction de données	3
3.1	La règle de majorité des 3/4	3
3.2	Le critère de Condorcet étendu	4
4	Résolution des sous instances	5
4.1	Programmation Dynamique	5
4.2	Programmation linéaire	6
5	Expérimentation	8
5.1	Instances réelles	8
5.2	Génération d'instances	8
6	Conclusion	8

1 Introduction

La théorie du vote s'intéresse aux règles qui permettent de trouver une décision collective à partir des préférences individuelles. Parmi les différentes méthodes existantes, l'agrégation de classement constitue un problème fondamental au cœur de la théorie du choix social, avec des applications allant de l'apprentissage automatique à la recherche web [1].

Parmi les règles de vote, la règle de Kemeny établit un classement de consensus qui minimise la distance totale de Kendall-tau (le nombre total de désaccords (paires inversées) entre le consensus et le profil de préférences).

D'un point de vue théorique, la règle de Kemeny possède des propriétés robustes, telles que le fait de satisfaire à la fois :

- le critère de Condorcet, c'est à dire que si un candidat, confronté à chaque autre candidat, est toujours gagnant (vainqueur de Condorcet), alors il sera classé premier dans le classement de Kemeny [2, 3].
- l'axiome de renforcement, qui stipule que si l'on considère deux groupes distincts de votants ayant les mêmes résultats, alors ce même résultat doit être obtenu lorsque l'on combine les votes des deux groupes [3].
- la neutralité qui garantit que si deux candidats échangent leurs positions dans les préférences de tous les votants, ils doivent également échanger leurs positions dans le classement final. Ainsi, tous les candidats sont traités de manière égale [3].

Problématique : cette solidité théorique est confrontée à une difficulté pratique majeure : sa complexité. En effet, le calcul d'un classement de Kemeny optimal est un problème NP-difficile [4, 5]. Ce résultat est valable même lorsque le nombre de votants est petit, pour $n = 4$, par exemple [5].

Afin de contrer cette problématique, des méthodes de décompositions ont été proposées. Ces méthodes réduisent l'instance originale à de plus petits sous-problèmes indépendants qu'il faudra ensuite résoudre.

Ce projet vise à implémenter des approches permettant de résoudre le problème de Kemeny de manière exacte et efficace, même sur des instances de grande taille. Ainsi l'objectif est de comparer les méthodes de résolution (programmation dynamique et programmation linéaire) ainsi que les méthodes de décomposition (le critère de Condorcet étendu [6] et la règle de majorité des 3/4 [4]). Afin d'obtenir des résultats d'évaluation réalistes, nous évaluerons ces méthodes sur des instances réelles stockées dans la bibliothèque Preflib. De plus, ce projet se concentre sur des préférences complètes et sans égalité.

2 Préliminaire

On considère un ensemble fini $C = \{1, \dots, m\}$ de m candidats et un ensemble $N = \{1, \dots, n\}$ de n votants. On note $L(C)$ l'ensemble des ordres linéaires, aussi appelés classements des candidats de C . Chaque votant $i \in N$ donne un classement $P_i \in L(C)$, appelé préférence de i . On note $P = \{P_1, P_2, \dots, P_n\}$, l'ensemble contenant les préférences des votants, aussi appelé profil de préférences.

La règle de Kemeny consiste à chercher un ordre linéaire $P^* \in L(C)$ qui minimise la somme des désaccord total des votants. Ce désaccord est mesuré par le score de Kemeny [4], c'est à dire la somme des distances de Kendall-Tau entre un rangement P_i et un profil de préférences P :

$$\Delta(P^*, P) = \sum_{i \in N} D_\tau(P_i, P^*)$$

où $D_\tau(P_i, P^*)$ est la distance de Kendall-Tau entre deux classements, définie de la manière suivante :

$$D_\tau(P_i, P^*) = \sum_{(a,b) \in C^2} \mathbf{1}_{\{a \succ b \text{ dans } P_i \text{ et } b \succ a \text{ dans } P^*\}}$$

Exemple 1. Si l'on considère une instance avec $m = 6$ candidats et $n = 8$ votants avec les préférences suivantes :

- $P_1 : a \succ c \succ f \succ e \succ d \succ b$
- $P_2 : a \succ c \succ f \succ e \succ d \succ b$
- $P_3 : a \succ c \succ e \succ f \succ d \succ b$
- $P_4 : a \succ d \succ c \succ f \succ e \succ b$
- $P_5 : a \succ d \succ c \succ f \succ e \succ b$
- $P_6 : a \succ b \succ d \succ c \succ f \succ e$
- $P_7 : a \succ b \succ f \succ e \succ d \succ c$
- $P_8 : a \succ b \succ f \succ e \succ d \succ c$

Pour déterminer le classement de Kemeny P^* , nous calculons la somme des distances de Kendall-Tau entre P^* et les préférences des votants. Dans le cas où P^* est égal à $a \succ c \succ e \succ f \succ d \succ b$, on a $D_\tau(P_1, P^*) = 1$ (seule la paire (e, f) est inversée).

On utilise également un graphe de majorité strict $G = (C, A)$, un graphe orienté dont les noeuds correspondent aux candidats, et un arc entre deux candidats a et b indique que le candidat a est préféré par plus de 50% des votants au candidat b .

La matrice de préférence est une matrice qui, pour chaque case (i, j) , comptabilise le nombre de votants préférant le candidat i au candidat j .

Nous avons choisi de développer ce projet en python.

Nous avons défini une classe *Instance* afin d'y stocker l'instance (une élection) sur laquelle nous évaluons nos méthodes. Une instance représente un profil de préférences sur un ensemble de candidats.

3 Réduction de données

La réduction de données vise à identifier et à fixer des relations d'ordre entre candidats qui sont garanties d'apparaître dans toute solution optimale. Cette réduction constitue une phase de prétraitement essentielle pour rendre la résolution d'instances de grande taille abordable.

3.1 La règle de majorité des 3/4

La règle de majorité des 3/4 repose sur la notion de préférence forte et permet d'identifier des candidats dont la position relative peut être déterminée sans ambiguïté.

Pour tout candidat a et b , on note $a \succ_{3/4} b$, si a est préféré à b par au moins 3/4 des votants [4]. Un candidat a est propre si et seulement si pour tout autre candidat b on a $a \succ_{3/4} b$ ou $b \succ_{3/4} a$ [4].

Les candidats non propres nécessitent un traitement plus approfondi, car leur position relative reste incertaine même après application de la règle de majorité des 3/4. Les candidats non propres d'une même position constituent une sous-instance. Le classement final de Kemeny s'obtient en concaténant les classements résolus des sous-instances et des candidats propres [4].

L'algorithme de la règle des 3/4 identifie d'abord les candidats propres. Puis, il établit un ordre complet sur les candidats.

Lemme 1 (Betzler, Bredereck et Niedermeier [4]). Soit $a \in C$ un candidat propre pour la règle de s -majorité et $b \in C \setminus \{a\}$ avec $3/4 \leq s \leq 1$. Si $a \succ_s b$, alors dans tout classement de Kemeny

a est classé avant b ($a \succ b$); si $b \succ_s a$, alors dans tout classement de Kemeny b est classé avant a ($b \succ a$).

La règle de majorité des 3/4 s'appuie sur le Lemme 1 pour ordonner les candidats propres. On connaît également l'ordre relatif entre chaque candidat propre et chaque autre candidat non propre.

Afin d'implémenter cette règle de décomposition, on vérifie si le premier candidat est propre. Si le candidat est propre, on applique la règle de majorité des 3/4 sur les candidats classés avant et sur les candidats classés après. S'il ne l'est pas, on le place à la fin de la liste des candidats et on applique la règle de majorité des 3/4 sur cette nouvelle liste.

Exemple 2. On considère le profil de préférences de l'exemple 1.

Les candidats propres sont : **a**.

Les candidats non propres sont : **b, c, d, e, f**.

On teste si le candidat **a** est propre. Comme **a** est propre, on applique la règle des 3/4 sur l'ensemble des candidats avant **a** ($\{\}$) et après (**b, c, d, e, f**).

Pour l'ensemble vide, la solution est triviale.

On teste ensuite si **b** est propre. Il ne l'est pas, donc on le place en fin de liste et on applique la règle des 3/4 sur l'ensemble **{c, d, e, f, b}**. On boucle sur cet ensemble car tous les candidats sont non propres. La variable essais conditionne l'arrêt et empêche une exécution infinie de la boucle. En effet, lorsque essais atteint le nombre de candidats, l'algorithme a fait le tour de tous les candidats; il ne reste donc plus que des candidats non propres dans l'ensemble.

Ainsi, le classement optimal partiel est : **a** $\succ$ **{b, c, d, e, f}**

3.2 Le critère de Condorcet étendu

Le critère de Condorcet étendu [6] constitue une version plus robuste du critère de Condorcet, qui se concentre sur un unique vainqueur.

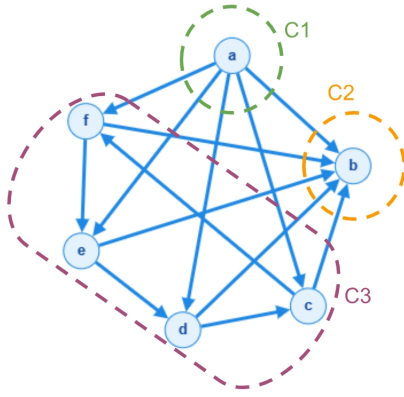
Le principe est le suivant : si un sous-ensemble de candidat C_α est majoritairement préféré à tous les candidats de l'ensemble complémentaire $C_\beta = C \setminus C_\alpha$ alors pour tout $x \in C_\alpha$ et $y \in C_\beta$, x est préféré à y par plus de la moitié des électeurs. Ainsi cet ensemble C_α doit se placer en tête de tout classement optimal. Ce principe peut être généralisé avec plusieurs sous-ensembles formant une partition de l'ensemble des candidats C . Le classement de Kemeny sur C est obtenu en juxtaposant les classements de Kemeny trouvés sur les sous-ensembles d'une partition.

L'algorithme du critère de Condorcet étendu consiste à construire le graphe de majorité stricte, puis d'ordonner les composantes fortement connexes.

Afin d'implémenter cet algorithme, nous avons utilisé une classe Graphe interne à la classe *Instance*. Un graphe est défini par une liste de noeuds et un dictionnaire d'adjacence. Cette classe permettra, par la suite, de créer le graphe de majorité d'une instance. Ce graphe peut être affichée grâce à la fonction *afficher_graphe*, qui utilise la bibliothèque *networkx*; ce qui nous permet de vérifier les résultats visuellement.

Pour identifier et ordonner les composantes fortement connexes, nous avons implémenté l'algorithme de Tarjan dans la fonction *condorcet_etendu*. Une composante fortement connexe de taille > 1 représente un groupe incohérent, soit une sous-instance à résoudre. Par conséquent, pour chacune d'entre elles, on crée un objet de la classe *Instance*.

Exemple 3. On considère le profil de préférences de l'exemple 1.



Le graphe de majorité stricte construit est celui ci-contre. On ordonne les composantes fortement connexes de la manière suivante : $C_1 \succ C_3 \succ C_2$.

Remarque : le critère de Condorcet étendu réduit plus l'instance que la règle de majorité des 3/4.

4 Résolution des sous instances

Les règles de décomposition transforment le problème initial NP-difficile en un ensemble de sous-problèmes indépendants de tailles beaucoup plus petites. Ces sous-problèmes doivent ensuite être résolus. Parmi les algorithmes utilisés, la programmation dynamique s'est avérée être particulièrement efficace pour les instances où le nombre de candidats est faible.

4.1 Programmation Dynamique

L'algorithme de programmation dynamique calcule le score de Kemeny (la somme des distances de Kendall-Tau) pour chaque sous-ensemble possible de candidats $C' \subseteq C$ et renvoie la solution qui minimise le score total.

Décomposition récursive [7] :

Cas de base : pour les sous-ensembles C' de taille 1, le calcul d'un classement de Kemeny est trivial. Il y a un seul candidat c et le score de Kemeny associé est de 0.

Étape récurrente : pour les sous-ensembles C' de taille $|C'| > 1$, l'algorithme teste itérativement chaque candidat $c \in C'$ comme étant celui placé en première position du classement local.

Calcul du score : le score total pour c est : $\text{score}(c, C') = \text{cout}(c, C') + \text{score}(C'')$. Le $\text{score}(C'')$ est le score optimal déjà calculé pour l'ensemble des candidats restants C'' et le $\text{cout}(c, C')$ est le coût engendré par le placement de c en tête du classement C' (nombre d'insatisfactions).

Optimisation : le candidat c qui minimise le score total, est donc placé en première position dans le classement optimal pour ce sous-ensemble C' .

En procédant ainsi, l'algorithme assure que le score final $\text{score}(C)$ est le score de Kemeny optimal pour l'ensemble complet C , et le chemin suivi pour l'obtenir reconstitue le classement de Kemeny optimal.

Pour implémenter cet algorithme, nous avons écrit une fonction itérative, *resolution_dyn*. Elle retourne un dictionnaire associant à chaque sous-ensemble de candidats un couple (candidat en tête, score). Pour chaque taille $t > 1$ et pour chaque combinaison de candidats, nous testons chaque élément en première position du classement afin de ne conserver dans le dictionnaire que celui qui minimise le score. Dans le cas particulier où $t=1$, l'unique candidat est directement enregistré.

Ce dictionnaire permet ensuite de reconstruire le classement final via la fonction *reconstruction_classement_PDyn*. Cette dernière extrait successivement les candidats de tête pour aboutir à la liste ordonnée finale.

Exemple 4. Application de la Programmation Dynamique

Soit l'instance réduite par le critère de Condorcet étendue dans l'exemple 3. Soit dict un dictionnaire vide avec pour clé l'ensemble des candidats ordonnés et pour valeur le couple (candidat en tête, score).

Pour chaque sous-ensemble de taille 1 ($\{c\}$, $\{d\}$, $\{e\}$, $\{f\}$), on ajoute le couple ($\{\text{candidat}\}$, ($\{\text{candidat}\}, 0$)).

Sous-ensembles de taille 2 : On calcule le score de Kemeny optimal pour le sous-ensemble. Par exemple, pour le sous-ensemble $\{c, d\}$:

- $c \succ d$ engendre un coût de 5,
- $d \succ c$ engendre un coût de 3.

Le score minimal est donc 3 pour ce sous ensemble. On ajoute dans dict , le couple ($\{c, d\}$, ($\{d\}, 3$)). On fait de même pour tous les autres.

Sous-ensembles de taille 3 : On calcule le score de Kemeny optimal pour le sous-ensemble, à partir des scores des sous-ensembles de taille inférieure. Par exemple, pour le sous-ensemble $\{c, d, e\}$:

- $c \succ \{d, e\}$ engendre un coût de $7 + 3 = 10$,
- $d \succ \{c, e\}$ engendre un coût de $8 + 2 = 10$,
- $e \succ \{c, d\}$ engendre un coût de $9 + 3 = 12$.

De même, on met à jour dict avec le nouveau couple ($\{c, d, e\}$, ($\{c\}, 10$)) (car c est le premier candidat testé, et on met à jour le candidat en tête si et seulement le score est strictement inférieur). On fait de même pour tous les autres.

Sous-ensembles de taille 4 : De manière identique, on obtient :

- $c \succ \{d, e, f\}$ engendre un coût de $9 + 7 = 16$,
- $d \succ \{c, e, f\}$ engendre un coût de $13 + 5 = 18$,
- $e \succ \{c, d, f\}$ engendre un coût de $16 + 10 = 26$.
- $f \succ \{c, d, e\}$ engendre un coût de $10 + 10 = 20$.

On met à jour dict avec le nouveau couple ($\{c, d, e, f\}$, ($\{c\}, 16$)).

Le score de Kemeny minimal est de 16 et correspond au cas où le candidat c est placé avant les candidats d , e et f dans le classement final. De plus, le classement optimal pour $\{d, e, f\}$ est $f \succ \{d, e\}$ et pour $\{d, e\}$, $e \succ d$. Ainsi, le classement final est $c \succ f \succ e \succ d$.

4.2 Programmation linéaire

L'agrégation de classement de Kemeny nécessite que le classement final soit un ordre strict. Ce problème peut donc être modélisé par le programme linéaire en nombres entiers suivant :

$$\min \sum_{a,b} Q_{ab} \cdot x_{ba} + Q_{ba} \cdot x_{ab} \quad (1)$$

$$x_{ab} + x_{ba} = 1 \quad (2)$$

$$x_{ab} + x_{bc} + x_{ca} \geq 1 \quad (3)$$

$$x_{ab} \in \{0, 1\} \quad (4)$$

Ce programme linéaire vise à minimiser le score total de Kemeny et retourne une solution du problème d'agrégation de Kemeny [4].

Le modèle utilise les variables binaires $x_{ab} \in \{0, 1\}$ qui valent 1 si le candidat a est classé avant le candidat b dans le classement de Kemeny et les variables Q_{ab} représentant le nombre de fois où le candidat a est classé avant b dans le profil de préférences.

Les contraintes suivantes garantissent que les solutions x_{ab} représentent un ordre linéaire valide :

- Contraintes d'ordre strict (contrainte (2)) : cette contrainte assure que pour toute paire de candidats (a, b) , on ne peut pas avoir à la fois $a \succ b$ et $b \succ a$ dans le classement final.
- Contraintes de transitivité (contrainte (3)) : cette contrainte est essentielle car elle évite l'existence de cycle. En effet, pour chaque triplet de candidats distincts (a, b, c) , si $a \succ b$ et $b \succ c$ alors on a forcément $a \succ c$.

Nous avons donc implémenté une fonction *resolution_pl* qui, à l'aide de Gurobi, résout ce programme linéaire. Pour ce faire, nous avons créé un modèle, en utilisant la matrice de préférences stockée dans l'instance (attribut *matPref*) pour définir les coefficients de la fonction objectif. Nous avons ensuite implémentée la fonction *reconstruction_classement_PL* qui, à partir du modèle résolu, renvoie le classement correspondant sous la forme d'une liste, ou lève une exception si aucune solution n'a été trouvée.

Exemple 5. : *Application de la Programmation Linéaire*

Pour l'instance de l'exemple 1, la matrice de préférences Q est la suivante :

$$Q = \begin{pmatrix} & \textcolor{red}{a} & \textcolor{green}{b} & \textcolor{blue}{c} & \textcolor{orange}{d} & \textcolor{purple}{e} & \textcolor{brown}{f} \\ \textcolor{red}{a} & 0 & 8 & 8 & 8 & 8 & 8 \\ \textcolor{green}{b} & 0 & 0 & 3 & 3 & 3 & 3 \\ \textcolor{blue}{c} & 0 & 5 & 0 & 3 & 6 & 6 \\ \textcolor{orange}{d} & 0 & 5 & 5 & 0 & 3 & 3 \\ \textcolor{purple}{e} & 0 & 5 & 2 & 5 & 0 & 1 \\ \textcolor{brown}{f} & 0 & 5 & 2 & 5 & 7 & 0 \end{pmatrix}$$

Après réduction par le critère de Condorcet étendu dans l'exemple 3, il reste à résoudre la sous-instance contenant les candidats $\textcolor{blue}{c}, \textcolor{orange}{d}, \textcolor{purple}{e}$ et $\textcolor{brown}{f}$.

La fonction objective est donc :

$$\min (3 \cdot x_{\textcolor{blue}{c}\textcolor{orange}{d}} + 5 \cdot x_{\textcolor{orange}{d}\textcolor{purple}{e}}) + (6 \cdot x_{\textcolor{purple}{e}\textcolor{brown}{f}} + 2 \cdot x_{\textcolor{brown}{f}\textcolor{purple}{e}}) + \dots + (1 \cdot x_{\textcolor{brown}{f}\textcolor{blue}{c}} + 7 \cdot x_{\textcolor{blue}{c}\textcolor{brown}{f}})$$

On définit alors les contraintes d'ordre strict, telles que :

$$\begin{aligned} x_{\textcolor{blue}{c}\textcolor{orange}{d}} + x_{\textcolor{orange}{d}\textcolor{blue}{c}} &= 1 \\ x_{\textcolor{purple}{e}\textcolor{brown}{f}} + x_{\textcolor{brown}{f}\textcolor{purple}{e}} &= 1 \\ x_{\textcolor{blue}{c}\textcolor{purple}{e}} + x_{\textcolor{purple}{e}\textcolor{blue}{c}} &= 1 \\ &\dots \\ x_{\textcolor{brown}{f}\textcolor{blue}{c}} + x_{\textcolor{blue}{c}\textcolor{brown}{f}} &= 1 \end{aligned}$$

On définit également les contraintes de transitivité. Par exemple :

$$\begin{aligned} x_{\textcolor{blue}{c}\textcolor{orange}{d}} + x_{\textcolor{orange}{d}\textcolor{purple}{e}} + x_{\textcolor{purple}{e}\textcolor{blue}{c}} &\geq 1 \\ x_{\textcolor{orange}{d}\textcolor{purple}{e}} + x_{\textcolor{purple}{e}\textcolor{brown}{f}} + x_{\textcolor{brown}{f}\textcolor{orange}{d}} &\geq 1 \\ &\dots \\ x_{\textcolor{brown}{f}\textcolor{blue}{c}} + x_{\textcolor{blue}{c}\textcolor{brown}{f}} + x_{\textcolor{brown}{f}\textcolor{purple}{e}} &\geq 1 \end{aligned}$$

Le solveur trouve alors la solution qui minimise la fonction objectif tout en respectant ces contraintes de classement.

5 Expérimentation

Afin d'évaluer les méthodes implémentées, nous avons utilisé deux méthodes d'évaluation mesurant différentes métriques, le temps d'exécution et la taille de la plus grande sous-instance (*eval_reduction*). La fonction *eval_temps* calcule le temps d'exécution d'une instance avec chacune des méthodes de décomposition, en combinant les deux et sans réduction.

On teste nos méthodes sur deux types d'instances, des instances réelles (*preflib*) et générées (avec le modèle de *Mallows*).

5.1 Instances réelles

Pour cette partie, nos instances sont issues de la bibliothèque *preflibtools*, et sont stockées dans des fichiers .soc. Nous avons donc récupéré toutes les instances sur le github PrefLib : <https://github.com/PrefLib/PrefLib-Data/tree/main/datasets>

Ainsi, la classe *Instance* contient une méthode de lecture de ces fichiers. Pour ce faire, nous utilisons les méthodes prédéfinies dans la bibliothèque *preflibtools.instances*. Ensuite, on construit la matrice de préférences de cette instance à l'aide de la méthode *comptage*. Dans cette matrice, on associe la case (i,j) au nombre de votants préférants le candidat i au candidat j.

5.2 Génération d'instances

explication de mallows

6 Conclusion

Références

- [1] Dwork Cynthia, Kumar Ravi, Moni Naor, and D. Sivakumar. Rank aggregation methods for the web. 2001.
- [2] Davenport Andrew and Kalagnanam Jayant. A computational study of the kemeny rule for preference aggregation. 2004.
- [3] Young H. P. and Levenglick A. A consistent extension of condorcet's election principle. 1978.
- [4] Nadja Betzler, Robert Bredereck, and Rolf Niedermeier. Theoretical and empirical evaluation of data reduction for exact kemeny rank aggregation. 2013.
- [5] Fischer Felix, Hudry Olivier, and Niedermeier Rolf. Weighted tournament solutions. 2016.
- [6] Truchon Michel. An extension of the condorcet criterion and kemeny orders. Technical report, 1999.
- [7] Nadja Betzler, Michael R. Fellows, Jiong Guo, Rolf Niedermeier, and Frances A. Rosamond. Fixed-parameter algorithms for kemeny rankings. 2009.